Disclaimer:

Dieses Material ist lizensiert.

Lizenz: CC BY-NC-ND 4.0 (wenn nicht anders angegeben)

- **Attribution (BY)**
- **Non Commercial (NC)**
- **No Derivatives (ND)**
- **No Warranty**
- **No Liability**

Einzelne Beispiele oder seperat geteilte Dokumente könnten einer anderen Lizenz unterliegen.

Autor: © Korobov Viktor, 2026

Netzwerke:

- LinkedIn
- GitHub
- YouTube
- LeetCode
- Steam

WIP:

Dieses Kursmaterial ist ein lebendiges Dokument und wird mit einem KI Assisstenten gewartet.

Letzter Stand: 22.10.26.

Gebt aufmerksam Acht auf das Gesagte!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen

Datenbankentwurf - Physischer Entwurf

Der physische Entwurf beschreibt die konkrete Umsetzung des logischen Modells eines Informationssystems auf eine spezifische Zielumgebung. Ziel ist es, das System so zu gestalten, dass es unter Berücksichtigung von Performance, Speicherplatz, Sicherheitsanforderungen und technischer Infrastruktur effizient betrieben werden kann.

Während der logische Entwurf technologieunabhängig ist und sich auf die Struktur und Beziehungen von Daten konzentriert, richtet sich der physische Entwurf auf die tatsächliche Implementierung innerhalb einer bestimmten Systemumgebung.

SQL

SQL ist eine standardisierte Abfragesprache zur Verwaltung und Manipulation relationaler Datenbanken. Sie erlaubt das Erstellen, Ändern, Löschen und Abfragen von Daten sowie die Definition von Datenstrukturen und Zugriffsrechten.

SQL besteht aus folgenden fünf Hauptbestandteilen:

- Data Query Language (DQL)
- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Control Language (DCL)
- Transaction Control Language (TCL)

Jeder Bestandteil deckt einen spezifischen Aspekt des Datenbankmanagements ab.

Zentrale Anwendungsbereiche:

Kategorie	Beispielbefehl	Beschreibung
Datenabfrage (DQL)	`SELECT`	Abfrage von Daten aus Tabellen
Datenmanipulation (DML)	`INSERT`, `UPDATE`, `DELETE`	Hinzufügen, Ändern, Löschen von Daten
Datenstruktur (DDL)	`CREATE`, `ALTER`, `DROP`	Definition und Veränderung von Tabellen, Views etc.
Rechteverwaltung (DCL)	`GRANT`, `REVOKE`	Vergabe und Entzug von Nutzerrechten
Transaktionen (TCL)	`COMMIT`, `ROLLBACK`	Absicherung und Steuerung von Änderungen

DDL

Ermöglicht die Definition und Veränderung der Struktur von Datenbankobjekten.

- `CREATE` – erstellt Tabellen, Sichten (Views), Indizes etc.
- `ALTER` – ändert bestehende Objekte
- `DROP` – löscht Objekte dauerhaft

Beispiel (`CREATE`):

```sql
CREATE TABLE produkte (
  id INTEGER PRIMARY KEY,
  name VARCHAR(100),
  preis DECIMAL(10, 2)
);
```

- Erstellt eine neue Tabelle `produkte`
- Mit drei Spalten: `id`, `name`, `preis`
- `id` ist Primärschlüssel

Beispiel (`ALTER`):

```sql
ALTER TABLE produkte
ADD COLUMN lagerbestand INTEGER DEFAULT 0;
```

- Fügt der bestehenden Tabelle `produkte` eine neue Spalte hinzu
- Neue Spalte: `lagerbestand` vom Typ `INTEGER`
- Standardwert ist `0`

Beispiel (`ALTER` - löschen einer Spalte):

```sql
ALTER TABLE produkte
DROP COLUMN lagerbestand;

```

- Entfernt die Spalte `lagerbestand` vollständig aus der Tabelle

> **Achtung:**
>
> `DROP` -Befehle sind nicht rückgängig zu machen.
> Vor dem Einsatz sollte ein Backup vorhanden sein.

Beispiel (`ALTER` - ändern eines Typs):

```sql
ALTER TABLE `produkte` CHANGE `preis`
`preis` INT(10) NOT NULL DEFAULT 0;

/* in MySQL < 8.0:
ALTER TABLE produkte
MODIFY preis INTEGER;
*/
```

- Neuer Datentyp: `INTEGER` statt z. B. `DECIMAL(10, 2)`
- Nur möglich, wenn alle Werte in der Spalte mit dem neuen Datentyp kompatibel sind.

> **Info**
>
> Optional kann eine Typumwandlung mit `USING` angegeben werden:
> Dies erzwingt eine Konvertierung über einen angegebenen Ausdruck.

Beispiel (`ALTER` -hinzufügen eines Spalten-Constraints):

```sql
ALTER TABLE produkte
ALTER COLUMN lagerbestand SET DEFAULT 0;

/* in MySQL < 8.0:
ALTER TABLE produkte
MODIFY COLUMN lagerbestand INT DEFAULT 0;
```

```
7  */
```

- Setzt den Standardwert für neue Datensätze auf `0`
- In älteren Versionen muss ggf. `MODIFY` verwendet werden:

Beispiel (`ALTER`-hinzufügen eines Tabellen-Constraints):

```
1  ALTER TABLE produkte
2  ADD CONSTRAINT preis_check CHECK (preis >= 0);
```

- Prüft, dass der Preis nicht negativ ist

Beispiel (`ALTER`-hinzufügen eines Foreign-Key-Constraints):

```
1  ALTER TABLE bestellungen
2  ADD CONSTRAINT fk_kunde
3  FOREIGN KEY (kunden_id) REFERENCES kunden(id);
```

- Stellt sicher, dass `bestellungen.kunden_id` nur Werte enthält, die in `kunden(id)` existieren
- Für `FOREIGN KEY` muss die referenzierte Spalte indiziert sein (`PRIMARY KEY` oder `UNIQUE`) und es muss der gleiche Datentyp vorliegen.

Beispiel (`DROP`):

```
1  DROP TABLE produkte;
```

- Löscht die Tabelle `produkte` dauerhaft aus der Datenbank
- Alle enthaltenen Daten und Strukturen gehen verloren

Achtung:

`DROP`-Befehle sind nicht rückgängig zu machen.
Vor dem Einsatz sollte ein Backup vorhanden sein.

Danksagung:

Liebe Lernende, vielen Dank für die Aufmerksamkeit!

Liebe Veranstalter, vielen Dank für die Bühne!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen